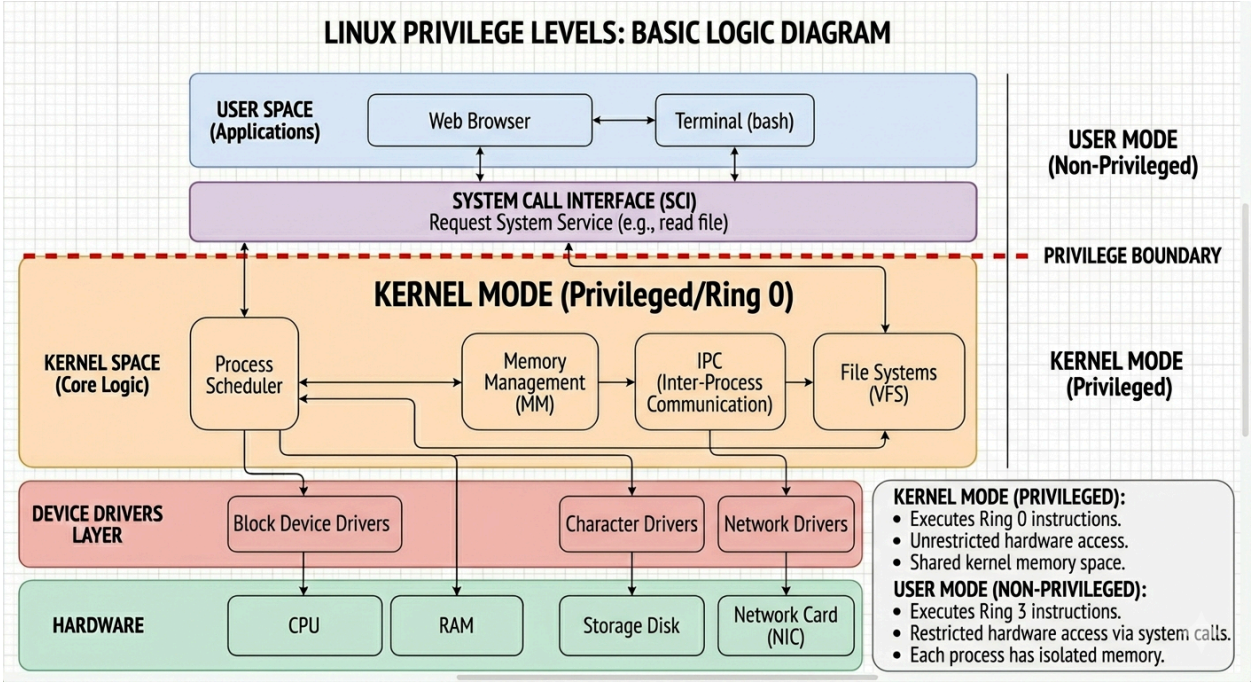
Name - Saikat Sheet
Batch - 25SUB5936_Linux System Programming
ID - 68500

Task 6:

- Privilege Levels Essay: Write a detailed essay explaining Linux's privilege levels, focusing on Kernel Mode and User Mode.
- Include examples of operations permitted at each level.



Understanding Linux Privilege Levels

In the Linux operating system, the stability and security of the entire system depend on a strict hierarchy of access known as **Privilege Levels**. This architecture is primarily divided into two distinct execution environments: **User Mode** and **Kernel Mode**. By separating where general applications run from where the core operating system logic resides, Linux prevents a single malfunctioning program from crashing the entire computer.

User Mode: The Restricted Playground

User Mode is the non-privileged execution mode where most software—such as web browsers, text editors, and your own custom scripts—operates. In this mode, the CPU enforces strict limitations on what the code can do. Programs running here are "sandboxed" in their own virtual memory space; they cannot directly access hardware or memory belonging to other processes.

Permitted Operations in User Mode:

- **Basic Computation:** Performing mathematical calculations or processing data within the application's allocated memory.
- **API Calls:** Requesting services from the system via high-level libraries (like `glibc`).
- **Variable Management:** Declaring and modifying strings, integers, or arrays that exist within the process's own local environment.

If a User Mode program attempts to access a hardware device directly or read a sensitive part of the system memory, the CPU generates a "protection fault," and the kernel typically terminates the process to protect the system.

Kernel Mode: The Command Center

Kernel Mode, also known as Supervisor Mode or "Ring 0," is the most privileged level of the system. The Linux Kernel—including the Scheduler, Memory Manager, and Device Drivers—runs at this level. In Kernel Mode, the CPU has unrestricted access to all hardware, all memory addresses, and every instruction the processor is capable of executing.

Permitted Operations in Kernel Mode:

- **Hardware Control:** Directly communicating with the CPU, RAM, disk controllers, and network interface cards.
- **Memory Management:** Managing the physical-to-virtual address translations and allocating "pages" of RAM to User Mode processes.
- **Interrupt Handling:** Pausing current tasks to respond to hardware signals, such as a keyboard press or data arriving via a network cable.

Because code in Kernel Mode has total control, a bug here (such as a faulty device driver) can result in a "Kernel Panic," causing the entire system to stop immediately to prevent data corruption.

The Gateway: System Calls

The transition between these two levels is handled by the **System Call Interface (SCI)**. When a User Mode program needs to perform a privileged task—like creating a new directory or sending data over the internet—it cannot do so itself. Instead, it executes a "system call."

The CPU then switches from User Mode to Kernel Mode, hands the request to the kernel, and waits. The kernel verifies if the user has the correct permissions (e.g., checking if the user owns the directory they are trying to modify), performs the task, and then switches the CPU back to User Mode to return the result to the application. This "gatekeeper" mechanism ensures that even though the kernel is all-powerful, it only exercises that power on behalf of authorized requests.
